

Usuarios persistentes y BCrypt

3. Usuarios persistentes y BCrypt

De dónde partimos

Al terminar el apartado anterior, tu API ya distinguía autenticación de autorización, pero utilizaba dos piezas deliberadamente provisionales: usuarios en `InMemoryUserDetailsManager` con contraseñas en texto claro (`{noop}`), y HTTP Basic enviándolas en cada petición, codificadas en Base64, no cifradas.

Hoy resuelves el primero: a partir de ahora los usuarios vivirán en PostgreSQL, con sus contraseñas protegidas mediante BCrypt. El segundo problema —las credenciales viajando en cada petición— lo resolverá JWT más adelante.

Esta pieza sí se mantendrá

A diferencia de los usuarios en memoria del apartado anterior, la persistencia en PostgreSQL y la protección de las contraseñas mediante BCrypt ya forman parte de la solución que mantendrás cuando sustituyas HTTP Basic por JWT.

Cifrar vs. hashear

	Cifrar	Hashear
Reversible	Sí, con la clave correcta	No, nunca
Para qué sirve	Proteger datos que necesitas recuperar tal cual	Proteger datos que solo necesitas comparar, nunca leer de vuelta

Una contraseña no necesitas nunca "leerla de vuelta": solo comprobar si coincide con la registrada. Por eso se hashean, no se cifran — ni siquiera tú, con acceso total a la base de datos, deberías poder recuperar la contraseña original.

El problema de un hash sin sal

Si simplemente aplicaras una función hash (`hash("miPassword123")`) a cada contraseña, dos usuarios con la misma contraseña tendrían el mismo hash guardado. Eso permitiría detectar que comparten contraseña y facilitaría ataques basados en hashes precalculados, como las tablas arcoíris.

Una tabla arcoíris es una colección precalculada que relaciona contraseñas habituales con los hashes que producen. Si un atacante roba una tabla de hashes, busca coincidencias directamente ahí, sin calcular cada contraseña desde cero.

La sal: mismo password, hashes distintos

La sal es un valor aleatorio distinto para cada contraseña, que se combina con ella antes de calcular el hash. Dos usuarios con la misma contraseña obtienen hashes completamente distintos:

Usuario	Contraseña	Sal	Hash guardado
ana	miPassword123	aleatoria, propia de ana	{bcrypt}\$2a\$10\$x7Ff...9kLp
luis	miPassword123	aleatoria, propia de luis	{bcrypt}\$2a\$10\$q2Rt...3mWx

Nadie que mire la tabla usuarios puede saber que ana y luis comparten contraseña.

Por qué BCrypt y no un hash rápido

Podrías pensar en usar SHA-256 directamente. El problema es justo su virtud en otro contexto: es rápido, pensado para calcularse millones de veces por segundo. Aplicado a contraseñas, eso juega en tu contra.

Un atacante con acceso a una tabla de hashes robada puede probar miles de millones de contraseñas por segundo contra un hash SHA-256, por fuerza bruta.

BCrypt: lento, a propósito

BCrypt está diseñado deliberadamente para ser lento, con un factor de coste configurable: insignificante para un login legítimo, pero devastador para un atacante que necesita probar millones de combinaciones.

Factor de coste	Tiempo aprox. por hash
4	~1 ms
10 (valor por defecto)	~60-100 ms
12	~250-400 ms

Cien milisegundos son imperceptibles para un login. Multiplicados por millones de intentos, la diferencia entre "factible" e "inviabile" se cuenta en años de cómputo.

La entidad Usuario

```
@Entity
@Table(name = "usuarios")
public class Usuario {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String username;

    @Column(nullable = false)
    private String password;

    @Enumerated(EnumType.STRING)
    private RolUsuario rol;

    private boolean activo = true;
}
```

El rol y la baja lógica

Territorio ya conocido de Acceso a Datos: una entidad JPA más, con su repositorio (UsuarioRepository, con un findByUsername añadido). La novedad es password: nunca contiene texto legible, solo el hash BCrypt.

```
public enum RolUsuario {  
    ADMIN,  
    USER  
}
```

activo es una baja lógica, no física: en vez de borrar la fila, la marcas como activo = false y la conservas — así no pierdes el histórico de qué creó o modificó ese usuario mientras tuvo cuenta.

El PasswordEncoder

```
@Bean
public PasswordEncoder passwordEncoder() {
    return PasswordEncoderFactories.createDelegatingPasswordEncoder();
}
```

Este mismo bean se usa tanto para codificar una contraseña al crear un usuario como para verificar una contraseña al hacer login (BCrypt no "descifra" el hash — vuelve a calcularlo con la contraseña recibida y compara).

DelegatingPasswordEncoder: un prefijo por algoritmo

`createDelegatingPasswordEncoder()` construye un `DelegatingPasswordEncoder`: antepone un prefijo al hash almacenado (por ejemplo, `{bcrypt}`) indicando qué algoritmo se usó — así, si en el futuro cambiaras de algoritmo, los hashes antiguos seguirían siendo reconocibles y verificables, sin migrar toda la tabla de golpe.

Anatomía de un hash BCrypt real

```
{bcrypt}$2a$10$N9qo8uLOickgx2ZMRZoMyeIjZAgcf17p92ldGxad68LJZdL17lhWy
```

Parte	Qué es
{bcrypt}	Prefijo añadido por DelegatingPasswordEncoder.
\$2a\$	Identifica la versión de BCrypt.
10\$	El factor de coste, cuyo trabajo aumenta exponencialmente.
el resto	Sal y hash codificados en una misma cadena — no hace falta guardar la sal aparte.

Un UserDetailsService propio

```
@Service
@RequiredArgsConstructor
public class BdUserDetailsService implements UserDetailsService {

    private final UsuarioRepository usuarioRepository;

    @Override
    public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
        Usuario usuario = usuarioRepository.findByUsername(username)
            .orElseThrow(() -> new UsernameNotFoundException("Usuario no encontrado: " + username));

        return User.withUsername(usuario.getUsername())
            .password(usuario.getPassword())
            .roles(usuario.getRol().name())
            .disabled(!usuario.isActivo())
            .build();
    }
}
```

El reemplazo directo de InMemoryUserDetailsManager

Implementa UserDetailsService, y en vez de buscar en una lista fija en memoria, busca en UsuarioRepository. Spring Security llama a loadUserByUsername(...) automáticamente; tú solo dices dónde encontrar al usuario. .disabled(!usuario.isActivo()) es justo lo que usa el activo de la entidad.



Cambiar el almacén de usuarios no obliga a modificar securityFilterChain

Sustituir InMemoryUserDetailsManager por BdUserDetailsService no obliga a tocar .csrf(...), .exceptionHandling(...), .httpBasic(...) ni las reglas ya existentes. Spring Security usa el UserDetailsService y el PasswordEncoder del contexto para autenticar.

El registro: RegisterRequestDTO

Todo lo de arriba explica cómo se verifica un usuario que ya existe — pero no cómo llega a existir el primero. Hace falta un endpoint público que reciba usuario y contraseña, codifique la contraseña, y guarde el Usuario resultante:

```
public record RegisterRequestDTO(  
    @NotBlank(message = "El nombre de usuario no puede estar vacío")  
    String username,  
    @NotBlank(message = "La contraseña no puede estar vacía")  
    String password  
) {}
```


AuthController

```
@RestController
@RequestMapping("/api/v1/auth")
@RequiredArgsConstructor
public class AuthController {

    private final UsuarioRepository usuarioRepository;
    private final PasswordEncoder passwordEncoder;

    @PostMapping("/register")
    public ResponseEntity<Void> register(@Valid @RequestBody RegisterRequestDTO dto) {
        Usuario usuario = new Usuario();
        usuario.setUsername(dto.username());
        usuario.setPassword(passwordEncoder.encode(dto.password()));
        usuario.setRol(RolUsuario.USER);
        usuarioRepository.save(usuario);
        return ResponseEntity.status(HttpStatus.CREATED).build();
    }
}
```

Quién codifica y quién compara

`passwordEncoder.encode(...)` genera aquí el hash que se guarda. Durante el login, `BdUserDetailsService` solo carga el usuario y devuelve ese hash; es `DaoAuthenticationProvider` quien utiliza el mismo `PasswordEncoder` para comparar la contraseña recibida con el hash almacenado.

Si alguien intenta registrar un username que ya existe, la restricción `unique = true` lo bloquea en la base de datos — el mismo `DataIntegrityViolationException` que tu `GlobalExceptionHandler` convierte en un 409 desde el primer apartado del tema.

El rol nunca lo decide quien se registra

⚠ Nunca dejes que el cliente decida sus propios privilegios

RolUsuario.USER está fijado aquí mismo, en el servidor — no viene del DTO que manda el cliente. Si RegisterRequestDTO incluyera un campo rol y lo guardaras tal cual, cualquiera podría registrarse a sí mismo como ADMIN con solo añadirlo al JSON.

Abriendo la ruta de registro

Como cualquier otra ruta nueva, hay que abrirla explícitamente. Añade esta regla antes de `anyRequest().authenticated()`, porque esa regla final captura todo lo que no haya coincidido antes:

```
.requestMatchers(HttpMethod.POST, "/api/v1/auth/register").permitAll()
```

Pero si nadie puede auto-asignarse ADMIN, ¿de dónde sale el primero? Ese usuario tiene que venir de fuera del propio flujo de registro.

El primer ADMIN: un ApplicationRunner

```
@Component
@RequiredArgsConstructor
public class UsuariosSeed implements ApplicationRunner {

    private final UsuarioRepository usuarioRepository;
    private final PasswordEncoder passwordEncoder;

    @Value("${libreria.admin.password}")
    private String adminPassword;

    @Override
    public void run(ApplicationArguments args) {
        if (usuarioRepository.findByUsername("admin").isEmpty()) {
            Usuario admin = new Usuario();
            admin.setUsername("admin");
            admin.setPassword(passwordEncoder.encode(adminPassword));
            admin.setRol(RolUsuario.ADMIN);
            usuarioRepository.save(admin);
        }
    }
}
```

Una vez en cada arranque

`ApplicationRunner` es una interfaz de Spring Boot con un único método, `run(...)`, que el framework ejecuta automáticamente una vez en cada arranque, después de crear el contexto y antes de aceptar tráfico. Basta con anotar `@Component` e implementar la interfaz — Spring Boot la detecta y la ejecuta sola.

El `if (...isEmpty())` es imprescindible: `run(...)` se ejecuta en cada arranque, no solo la primera vez. Sin esa comprobación, intentarías insertar el mismo admin cada vez que reinicies, y chocarías con la restricción `unique = true` del `username`.

Esa contraseña tampoco debería estar en tu repositorio

Aunque `admin123` ya no viva en ningún `.java`, si la escribieras tal cual en `application-dev.yml` seguirías teniendo el mismo problema: ese fichero se sube a Git. La solución es un fichero de propiedades aparte, que nunca se versiona:

```
# application-dev.yml (Sí se sube a Git)
spring:
  config:
    import: "optional:application-dev-local.yml"
```

El fichero local, fuera de Git

```
# application-dev-local.yml (en .gitignore, NO se sube a Git)
libreria:
  admin:
    password: admin123
```

```
# application-dev-local.yml.example (Sí se sube a Git – sin valores reales)
libreria:
  admin:
    password: pon-aqui-tu-propia-contraseña
```

El prefijo optional: indica que la ausencia del fichero no provoca un error por sí sola. Pero `@Value("${libreria.admin.password}")` exige que la propiedad exista en alguna fuente: si falta, la aplicación no arranca.

Ese fichero no existe fuera de tu máquina

⚠ Piénsalo cuando montes CI o un despliegue

Si no se sube a Git, no llega a ningún sitio por Git. Cualquier entorno que arranque tu aplicación desde el repositorio limpio —un pipeline de CI, un servidor de despliegue, el ordenador de un compañero— no tiene ese fichero, y la aplicación no arranca. En esos entornos tienes que inyectar cada valor por otra vía: una variable de entorno, un parámetro, o un "secreto" del propio sistema de CI.

Cada persona que clona el repositorio copia el .example, rellena sus propios valores, y ese fichero final se queda solo en su máquina — el mismo patrón que reutilizarás para el secreto de JWT.

Los usuarios ya no están definidos directamente en el código: viven en PostgreSQL y sus contraseñas se almacenan mediante hashes que no permiten recuperar el texto original. Pero sigue quedando el segundo problema del apartado anterior: HTTP Basic envía las credenciales en cada petición. En el próximo apartado, JWT permitirá autenticarse una vez y presentar un token en las peticiones siguientes.

Ideas clave

- Cifrar es reversible (con clave); hashear no lo es — las contraseñas se hashean, nunca se cifran.
- La sal hace que contraseñas iguales produzcan hashes distintos, evitando comparación directa y tablas arcoíris.
- BCrypt es deliberadamente lento (factor de coste configurable), resistente a fuerza bruta frente a un hash rápido como SHA-256 a secas.
- DelegatingPasswordEncoder antepone un prefijo (`{bcrypt}`) al hash, indicando el algoritmo usado — el mismo bean codifica y verifica.

Ideas clave (cont.)

- Un `UserDetailsService` propio sustituye al `InMemoryUserDetailsManager`: busca los usuarios en la base de datos real; cambiar el almacén no obliga por sí mismo a modificar `securityFilterChain`.
- Un endpoint de registro público crea usuarios reales, siempre con rol `USER` fijado en el servidor. El primer `ADMIN` no puede salir de ahí: hace falta un seed aparte (`ApplicationRunner`).
- Ese seed no debe llevar la contraseña escrita en el código ni en un fichero versionado: va en `application-dev-local.yml` (en `.gitignore`), importado con `spring.config.import: optional:...`, acompañado de un `.example` sin valores reales que sí se sube a Git.